Language Models and Agents

Individual Project Report - 2025201017 Aryan

1 Introduction

1.1 Problem Statement

The task for our entire project was to create two language models for two respective languages, one high resource and one low resource.

High resource language chosen was Hindi and low resource language chosen was Nepali.

1.2 Overall Objective

The overall objective was divided into 3 phases:

- **Phase 1:** Involved the task of creating the monolingual corpus & training tokenizers for both languages separately.
- **Phase 2:** Involved using the Transformer architecture and training it on the corpus we constructed in Phase 1.
- **Phase 3:** Involved fine-tuning the base pretraining models on a synthetic dataset to evaluate logical reasoning capabilities of the models.

Hindi and Nepali were chosen particularly because both the languages share devanagri scripts which allows for a good baseline for how resource availability affects a models performance. Hindi had the highest digital footprint making constructing the corpora easier compared to Nepali whose resources are far fewer.

The overall pipeline I followed is as following:

- **Data & Tokenization:** Raw web-scraped and downloaded text corpora were aggressively cleaned, deduplicated, and used to train independent 10,000-vocabulary SentencePiece BPE tokenizers.
- **Pretraining:** A custom 25.8M-parameter decoder-only Transformer was trained from scratch on the processed corpora.
- **Reasoning Fine-Tuning:** The pretrained models were fine-tuned on synthetically generated, target-language logic datasets (12,000 examples per language).
- **Evaluation & Analysis:** The fine-tuned models were evaluated on exact-match reasoning accuracy across both structurally familiar and strictly held-out templates. Finally, a post-finetuning attention analysis was conducted—comparing early and late layers before and after fine-tuning—to trace how the models' internal representational focus adapted to solve targeted reasoning tasks.

Now lets look at the overall picture of the project phase by phase.

2 Phase 1

The Hindi and Nepali workflow are completely independent. They do not share datasets, tokenizers, vocabularies, or model files. They only share general python configurations and utility function code.

The assignment target is approximately 500M training tokens per language, with at least 20% of the final tokens coming from manually collected material.

2.1 Data Acquisition

For both languages, we used the Indic HPLT v2 dataset as the primary downloaded backbone. To meet the 20% manual data requirement, we extracted Wikipedia and Wikisource dumps ourselves and scraped several news sites.

2.1.1 Hindi Sources

Table 1: Hindi Data Sources

Source	Category	Collection Method	Raw Documents	Raw Words
Indic HPLT v2 (hi)	Downloaded	HuggingFace dataset download	614,314	285,401,242
Hindi Wikipedia	Manual	Wikimedia XML dump extracted via custom script	168,299	51,803,550
Hindi Wikisource	Manual	Wikimedia XML dump extracted via custom script	167,471	52,805,749
Dainik Jagran	Manual	Web scraping (article body text)	30	12,308
Amar Ujala	Manual	Web scraping (article body text)	85	34,901
Live Hindustan	Manual	Web scraping (article body text)	119	73,708

2.1.2 Nepali Sources

Table 2: Nepali Data Sources

Source	Category	Collection Method	Raw Documents	Raw Words
Indic HPLT v2 (ne)	Downloaded	HuggingFace dataset download	966,131	285,524,100
Nepali Wikipedia	Manual	Wikimedia XML dump extracted via custom script	30,285	7,061,109
Nepali Wikisource	Manual	Wikimedia XML dump extracted via custom script	80	90,301
Onlinekhabar	Manual	Web scraping (article body text)	28,987	10,489,745
Ekantipur	Manual	Web scraping (article body text)	29,088	12,373,873
Ratopati	Manual	Web scraping (article body text)	7,245	2,177,890
Setopati	Manual	Web scraping (article body text)	2,581	284,433
Kantipur	Manual	Web scraping (article body text)	90	39,202
Nepali Archives	Manual	Web scraping (literary works)	4	6,002
E-Pustakalaya	Manual	Catalog metadata	4	2,134

The overall methodology involves the follwing 4 steps:

2.2 Preprocessing

We cleaned the raw data so that it can actually be used in training. It was processed sequentially using streaming jsonl readers so as to not overwhelm memory usage. Every document went through 5 stages of preprocessing:

1. **HTML Removal:** USing regex based functions the useless HTML tags were removed from the documents that were manually collected form online sites.
2. **Unicode Normalization:** NFKC normalization to standardize character representations. Some characters in both the languages can be written in multiple combinations of other characters so this would end up bloating the vocab size if there are differing unicode representation for same characters. NFKC strips everything down and then merges them togethor based on if it exists in their unicode library.
3. **Whitespace Normalization:** Multiple spaces and tabs were collapsed into single spaces, but paragraph boundaries were preserved.
4. **Filtering:** Documents shorter than 50 characters were removed.
5. **Content Preservation:** We retained digits and legitimate multilingual content (e.g., Latin-script names and technical terms). For Nepali, an aggressive Devanagari-ratio filter was considered but ultimately rejected because it inadvertently removed valid news articles containing English loanwords or numbers.

During processing, provenance metadata (doc_id, source_name, language) was preserved for every document.

2.3 Deduplication

Deduplication was performed globally across all sources for each language.

Leading and trailing whitespaces were first stripped from each documents.

A SHA-256 hash was then generated for the exact text.

These hashes were then maitained in a seen or not seen set. If a hash was already seen it was discarded.

We processed manual sources before downloaded sources. This priority ordering ensured that if a document appeared in both a scraped news site and HPLT, the manual source's version (with correct provenance) was retained.

2.4 Splitting

We used a deterministic document-level 90/5/5 split for train, validation, and test sets. We used a fixed random seed (42). The random generator was seeded per source using `f"seed_{source_name}"` so that the split remains stable regardless of the order in which sources are processed.

2.5 Deduplication and Splitting Statistics

The Hindi corpus contained 950,318 raw documents. After preprocessing and global deduplication, 948,012 documents remained (a reduction of 2,306 documents). A dedicated Hindi deduplication report file was not generated, so these numbers are derived from the raw inspection and final statistics.

The deterministic 90/5/5 split produced:

- Train: 853,505 documents (351,201,146 words)
- Validation: 47,496 documents (19,464,204 words)
- Test: 47,011 documents (19,328,415 words)

2.6 Tokenizer Training

We trained a SentencePiece BPE tokenizer for Hindi. While 8K and 10K vocabulary candidates were evaluated during the pipeline, the specific candidate comparison metrics were not retained in the repository. We selected the 10,000 vocabulary size based on those initial evaluations.

Similarly for nepali: We trained two SentencePiece BPE candidates on a reservoir sample of 2,330,280 sentences from the full training split.

Table 3: Nepali Tokenizer Candidates

Candidate	Vocab Size	Val Tokens	Fertility	Chars/Token
nepali_bpe_8000	8,000	26,096,966	1.636	3.949
nepali_bpe_10000	10,000	25,067,886	1.571	4.111

The UNK percentages for the two candidates were 2.104% and 2.191%, respectively.

The final tokenizer (hindi_bpe_10k_final) was trained on a deterministically sampled subset of the training data.

The final Hindi corpus easily exceeds the minimum requirements.

- Algorithm: SentencePiece BPE
- Vocabulary Size: 10,000
- Total Actual BPE Tokens: 587,927,125
- Fertility: 1.508 tokens per word
- UNK Percentage: 2.576%

Manual vs. Downloaded Breakdown (by Actual Tokens):

- Manual: 176,068,967 tokens (29.95%)
- Downloaded (HPLT): 411,858,158 tokens (70.05%)

The manual token proportion of 29.95% satisfies the $\geq$20% requirement.

2.7 Nepali Full-Corpus Model

We trained the selected 10K BPE tokenizer on a larger deterministic sample of the full corpus (up to 5 million sentences per source). We applied this tokenizer (nepali_bpe_10k_final) to the complete Nepali corpus (1,061,336 documents, 317,069,245 words).

- Actual BPE tokens: 496,534,521
- Fertility: 1.566
- UNK Percentage: 2.162%

The Composition Issue: Under this tokenizer, the manual data accounted for only 10.14% of the actual BPE tokens, because the downloaded HPLT corpus was much larger.

The instructor Q&A permitted using a 1 word = 3 tokens heuristic for evaluating corpus composition. Under this heuristic, the full corpus contained approximately 21.1% manual data. However, to strictly satisfy the 20% manual requirement under both the heuristic and the actual tokenizer output, we created a second dataset.

We retained nepali_bpe_10k_final and the full corpus as a high-data experimental baseline, but it is not our official assignment submission model.

2.8 The Assignment-Compliant Model (nepali_bpe_10k_assignment)

To increase the manual token proportion, we deterministically downsampled the HPLT corpus (from ∼285M words to ∼118M words) while keeping all manual data unmodified. We wrote these downsampled files to nepali/data/final/.

We then trained a new tokenizer (nepali_bpe_10k_assignment) on this assignment-compliant corpus.

Final Assignment-Compliant Statistics:

- Total Words: 149,592,666
- Algorithm: SentencePiece BPE
- Vocabulary Size: 10,000
- Fertility: 1.570
- UNK Percentage: 2.181%

Manual Proportion (Heuristic vs Actual):

- Instructor Heuristic (1 word = 3 tokens): 448,777,998 total tokens. 94,777,086 manual tokens (21.12% manual).
- Actual BPE Tokenizer Output: 234,800,139 total actual tokens. 50,248,944 manual actual tokens (21.40% manual).

Note: The assignment target is approximately 500M tokens. Using the permitted heuristic, this corpus contains ∼448.78M tokens, meeting the target. The actual BPE token count of ∼234.8M is lower because Nepali SentencePiece fertility (1.570) is much lower than the heuristic assumption of 3.

This downsampled corpus and the nepali_bpe_10k_assignment tokenizer is what I consider my final submission and what was ultimately used for the next two phases.

2.9 Final Comparison

The following table compares the Hindi corpus against the final assignment-compliant Nepali corpus.

Table 4: Final Corpus Comparison

Metric	Hindi	Nepali (Assignment)
Total documents	948,012	494,642
Total words	389,993,765	149,592,666
Manual words	104,664,750	31,592,362
Downloaded words	285,329,015	118,000,304
Instructor heuristic token estimate	—	∼448.78M
Actual BPE tokens	587,927,125	234,800,139
Manual token percentage (actual)	29.95%	21.40%
Tokenizer algorithm	SentencePiece BPE	SentencePiece BPE
Vocabulary size	10,000	10,000
Fertility	1.508	1.570
UNK percentage	2.576%	2.181%

Drive link for all phase 1 artifacts:

https://drive.google.com/drive/folders/1IfzywcoVXSYJI_VuljQMAH7Qhwka3CTy?usp=drive_link

3 Phase 2

This is the phase were we designed the decoder only transformer architecture. It was primarily chosen because it is the industary standard for modern language models and because it was fundamentally desaigned for autoregressive tasks which is perfectly suited to general text generation.

3.1 Architecture

Table 5: Transformer Architecture

Hyperparameter	Value
Vocabulary Size	10,000
Hidden Dimension (d_{model})	448
Number of Transformer Blocks	8
Attention Heads (n_{heads})	7
Head Dimension (d_k)	64
FFN Hidden Dimension (d_{ff})	2048
Context Length	512 tokens
Positional Representation	Learned absolute positional embeddings
Normalization	Pre-LayerNorm
Activation Function	GELU
Attention Mechanism	Standard multi-head self-attention with causal mask
Dropout	0.1
Weight Tying	Yes (LM head shares weights with token embedding)

3.2 Intrinsic Evaluation

Table 6: Intrinsic Evaluation Metrics

Metric	Hindi (Validation)	Nepali (Validation)
Validation Loss	4.1015	4.1790
Perplexity	60.4292	65.2973
Bits Per Byte (BPB)	0.7017	0.5435

Comparison: Hindi performs slightly better on validation loss and perplexity, but Nepali exhibits a lower Bits Per Byte (BPB). Since lower BPB indicates fewer bits required to encode each byte, the Nepali model had better byte-level compression under this metric. However, BPB should be interpreted alongside the other metrics because the two corpora and tokenization characteristics differ.

3.3 Generation Evaluation

Generation evaluation was used to measure the quality of the text generated by the model.

3.3.1 Hindi Generation Metrics

Table 7: Hindi Generation Metrics

Strategy	BLEU	chrF	Repetition Rate	Distinct-1	Distinct-2
greedy	4.1595	18.9905	0.8020	0.3652	0.4445
temp_0.5	4.9206	20.8806	0.6196	0.5931	0.7725
temp_1.0	5.1846	20.5922	0.1934	0.8912	0.9952
temp_1.5	4.5572	21.2614	0.1189	0.9336	0.9951

3.3.2 Nepali Generation Metrics

Table 8: Nepali Generation Metrics

Strategy	BLEU	chrF	Repetition Rate	Distinct-1	Distinct-2
greedy	5.3712	21.0384	0.7307	0.4007	0.4501
temp_0.5	5.2468	23.6248	0.6351	0.5487	0.6441
temp_1.0	3.5921	26.4760	0.0950	0.9402	0.9918
temp_1.5	3.2390	23.8434	0.0457	0.9771	1.0000

3.4 Temperature Effects

Lower temperatures (greedy, 0.5) show higher repetition rates and lower distinct-N scores, meaning they suffer from degenerate repeating loops. As temperature increases (1.0, 1.5), repetition drastically decreases and distinct-N approaches 1.0 (higher diversity). However, BLEU drops significantly at temperature 1.5 as the generation becomes too random to match the reference text.

3.5 Attention Analysis

Finally attention analysis was done to establish a baseline for how well attention performs later on after finetuning and to measure mean attention distance and entropy, we could confirm that the models successfully learned expected structural behaviours.

Table 9: Phase 2 Attention Statistics

Language	Layer	Mean Entropy	Mean Distance
Hindi	Layer 0	1.9417	6.0668
Hindi	Layer 7	1.7901	8.1937
Nepali	Layer 0	2.2250	6.6935
Nepali	Layer 7	1.7572	8.0824

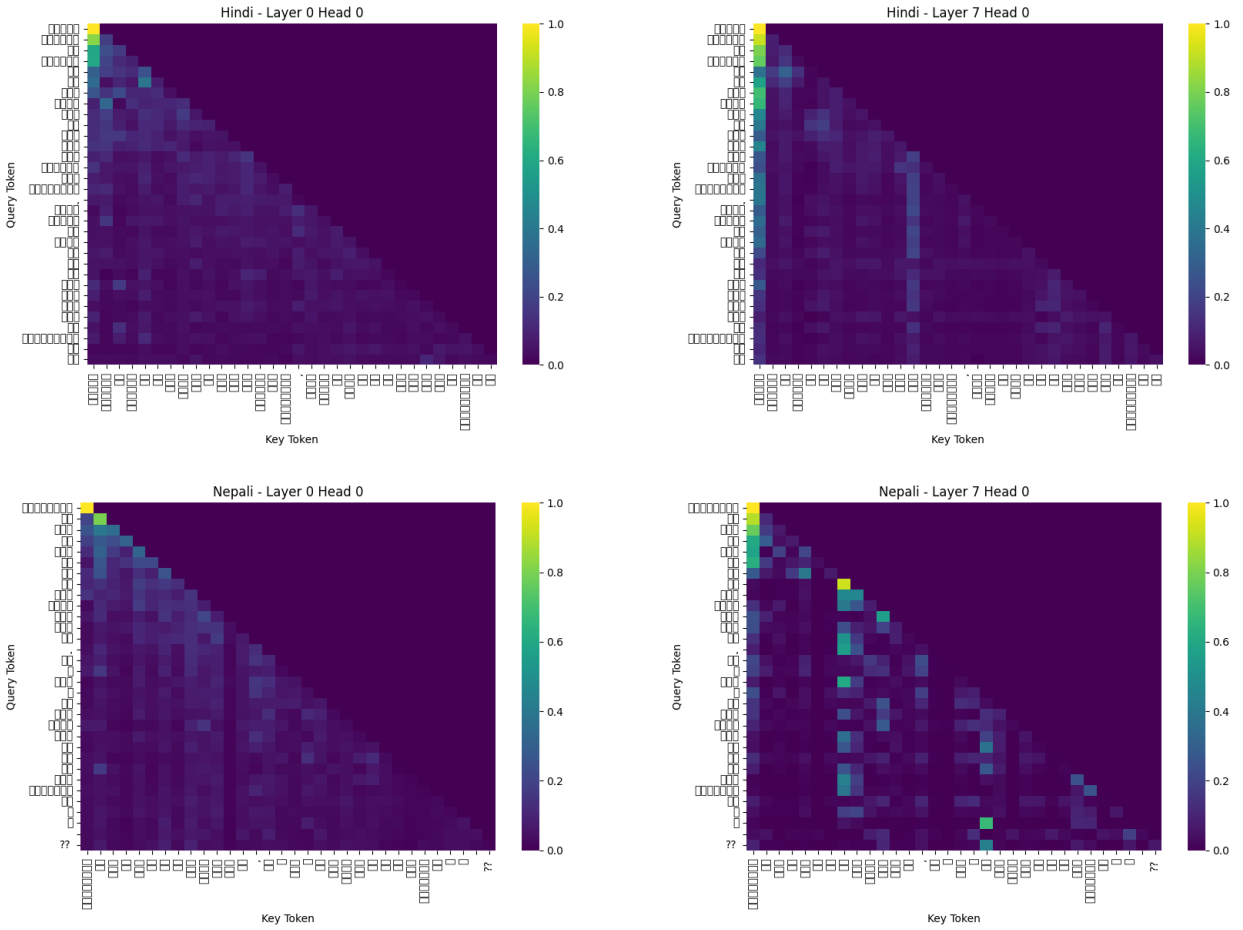


Figure 1: Phase 2 attention heatmaps for the Hindi and Nepali pretrained models.

3.5.1 Early vs. Late Layers

In both models, early layers (Layer 0) exhibit lower mean distance and higher entropy. Late layers (Layer 7) display higher mean distance and lower entropy. This indicates that attention became more concentrated while also spanning somewhat longer distances in the final layer.

3.5.2 Hindi vs. Nepali Comparison

The attention behavior is structurally similar across both models, suggesting the Transformer successfully learned similar core language-modeling mechanisms despite differences in corpus size and composition.

3.6 ROUGE-L

One generation metric I couldnt evalute the model was using ROUGE-L. The computed ROUGE-L score came out to exactly 0.0000 across all decoding strategies for both languages. This was because the standard rouge_score library utilizes an English-centric regex tokenizer ([a-z0-9]+) that aggressively strips all Devanagari characters. Consequently, it parses 0 tokens from both the generated and reference Indic strings, producing an artificial zero score.

Drive links for all the models:

https://drive.google.com/drive/folders/1J2CLalX8e9tWaH7i1XbgtrmQq5HKG8Nxr?usp=sharing

4 Phase 3

This phase involved creating a synthetic finetuning dataset and then finetuning the model. Independent reasoning fine-tuning for Hindi and Nepali was conducted, leveraging the Phase 2 pretrained checkpoints (step 20,000) and maintaining the fixed tokenizers and vocabularies established in earlier phases. A post-finetuning attention analysis was then done to observe how internal attention patterns

changed as the models learned to perform defined reasoning tasks.

4.1 Reasoning Dataset

The dataset was synthetically generated and constructed directly in the target languages.

- **Total Samples:** 12,000 instances per language.
- **Splits:** 9,600 Train, 1,200 Validation, 1,200 Test.
- **Task Types:** Direct, Attribute, and Transitive.
- **Attributes:** Height, age, weight, generic_order, price, possession_count, length, speed.
- **Template Diversity:** Dozens of unique premise-question structural templates were mapped directly into target-language phrasing for Hindi and Nepali.
- **Unique Reasoning Signatures:** 10,452 unique reasoning signatures accumulated across the full Hindi dataset, and 10,432 unique reasoning signatures across the full Nepali dataset (note: these are aggregate full-dataset statistics, not test-set counts).
- **Tokenization / Sequences:** Sequences are short logic puzzles formulated as prompts terminating in “ : ” (or equivalent).

4.2 Leakage / Generalization Design

To evaluate the models' abstract generalization capabilities, we structurally segmented the dataset:

- **Train/Validation:** Contained only examples generated from “familiar” templates.
- **Test:** Contained instances generated from both familiar and strictly “held-out” templates.

There is zero underlying reasoning-instance leakage between the test split and the train/validation splits (the test set contains 809 genuinely unique reasoning instances in Hindi, and 789 in Nepali).

Test Distribution:

- Hindi Test: 621 familiar, 579 held-out.
- Nepali Test: 599 familiar, 601 held-out.

4.3 Fine-Tuning Method

Fine-tuning parameters were defined by `finetune_config_hindi.yaml` and `finetune_config_nepali.yaml`:

- **Optimizer:** AdamW (beta1=0.9, beta2=0.95, eps=1.0e-8) with weight_decay=0.1.
- **Learning Rate:** $5.0e-5$ (min_lr: $5.0e-6$), managed by a linear warmup and cosine decay scheduler (warmup_steps=300).
- **Batch Size:** 16 (with grad_accum_steps=1).
- **Steps & Duration:** 3,000 max_steps.
- **Context Length:** 512 tokens.
- **Dropout:** 0.1.

Loss Masking Methodology: The standard causal language modeling objective was retained without structural modifications. However, the model's loss calculation was heavily masked: all target positions corresponding to the prompt text were ignored. Only the positions generating the answer and the End-of-Sequence (EOS) token remained loss-bearing. This forces the model to learn the reasoning-to-answer mapping without wasting updates on reconstructing the prompt.

4.4 Reasoning Evaluation

The primary metric for this reasoning task is an exact-match evaluation on the 1,200 test set examples. The evaluator operates using the following protocol:

- **Decoding:** Greedy decoding.
- **Extraction:** The evaluator extracts the first lexical word from the model's generation (stopping at spaces or punctuation).
- **Comparison:** This extracted word is checked for an exact string match against the ground-truth

answer.

- **EOS Handling:** As long as the first word is correct, an EOS token is not strictly required.
- **Scope:** This metric specifically measures synthetic exact-match reasoning and does not reflect general natural-language generation quality.

4.5 Reasoning Results

Table 10: Phase 3 Reasoning Results

Language	Model	Overall	Familiar	Held-out
Hindi	Pretrained	40/1200 (3.33%)	33/621 (5.31%)	7/579 (1.21%)
Hindi	Fine-tuned	559/1200 (46.58%)	475/621 (76.49%)	84/579 (14.51%)
Nepali	Pretrained	22/1200 (1.83%)	17/599 (2.84%)	5/601 (0.83%)
Nepali	Fine-tuned	495/1200 (41.25%)	451/599 (75.29%)	44/601 (7.32%)

Absolute Percentage-Point (pp) Improvement via Fine-Tuning:

- Hindi: +43.25 pp overall | +71.18 pp familiar | +13.30 pp held-out
- Nepali: +39.42 pp overall | +72.45 pp familiar | +6.49 pp held-out

Observation: Both models show drastic, > 71 pp improvements on familiar templates, reaching $\sim$75-76% accuracy. However, both models exhibit severe performance drop-offs on structurally held-out templates, yielding only single-digit to low double-digit improvements over their baselines.

Since we used binary exact match metrics to evaluate performance statistics, i didn't actually save what the model generated as text to a prompt. Since i dont have a record of above picking examples of successes and failure would hence be misleading and are therefore ommitted from the report.

4.6 Attention Analysis Method

To observe internal representational shifts, a post-finetuning attention analysis was conducted.

- **Methodology:** 3 familiar and 3 held-out examples were deterministically selected per language from the test split. Identical input token IDs were passed through the Pretrained (step 20,000) and Fine-Tuned (step 3,000) models.
- **Scope:** We extracted attention matrices from Layer 0 (early) and Layer 7 (late). Quantitative metrics were computed across all 7 heads, while Head 0 was used for primary heatmap visualization.
- **Metrics:** Definitions remain identical to Phase 2. Mean Attention Distance measures local vs. long-range behavior. A lower value indicates local, adjacent focus; a higher value indicates long-range attention spanning the sequence.
  Attention Entropy measures focus sharpness. A lower value indicates sharp, sparse attention; a higher value indicates diffuse, uniform attention.
- **Important point:** This analysis is descriptive only and does not claim that the above attention patterns prove a reasoning mechanism.

4.7 Attention Results

Aggregate metric shifts following fine-tuning indicate distinct layer-wise behaviors:

4.7.1 Early Layer (Layer 0)

- Hindi: Mean distance dropped slightly by -0.015, and entropy decreased by -0.036.
- Nepali: Mean distance increased slightly by +0.124, and entropy decreased by -0.020.

Observation: Early layers remained broadly stable and local, performing initial positional and lexical aggregation mostly unaffected by reasoning fine-tuning.

4.7.2 Late Layer (Layer 7)

- Hindi: Mean distance increased significantly (+0.391, becoming more long-range), and entropy dropped dramatically (-0.383, becoming sharper).
- Nepali: Mean distance dropped significantly (-0.460, becoming more local), and entropy dropped dramatically (-0.437, becoming sharper).

Observation: In both languages, the final layer showed a large drop in entropy, indicating that the attention became more concentrated after fine-tuning. These changes are consistent with the models developing different attention patterns after fine-tuning, although the attention analysis alone does not establish that these changes are directly responsible for the improved reasoning performance.

4.7.3 Head-Level Highlights

In the Hindi late layer, Head 4 entropy plummeted by -0.925.

In the Nepali late layer, Head 0 distance collapsed by -1.792 (becoming highly local).

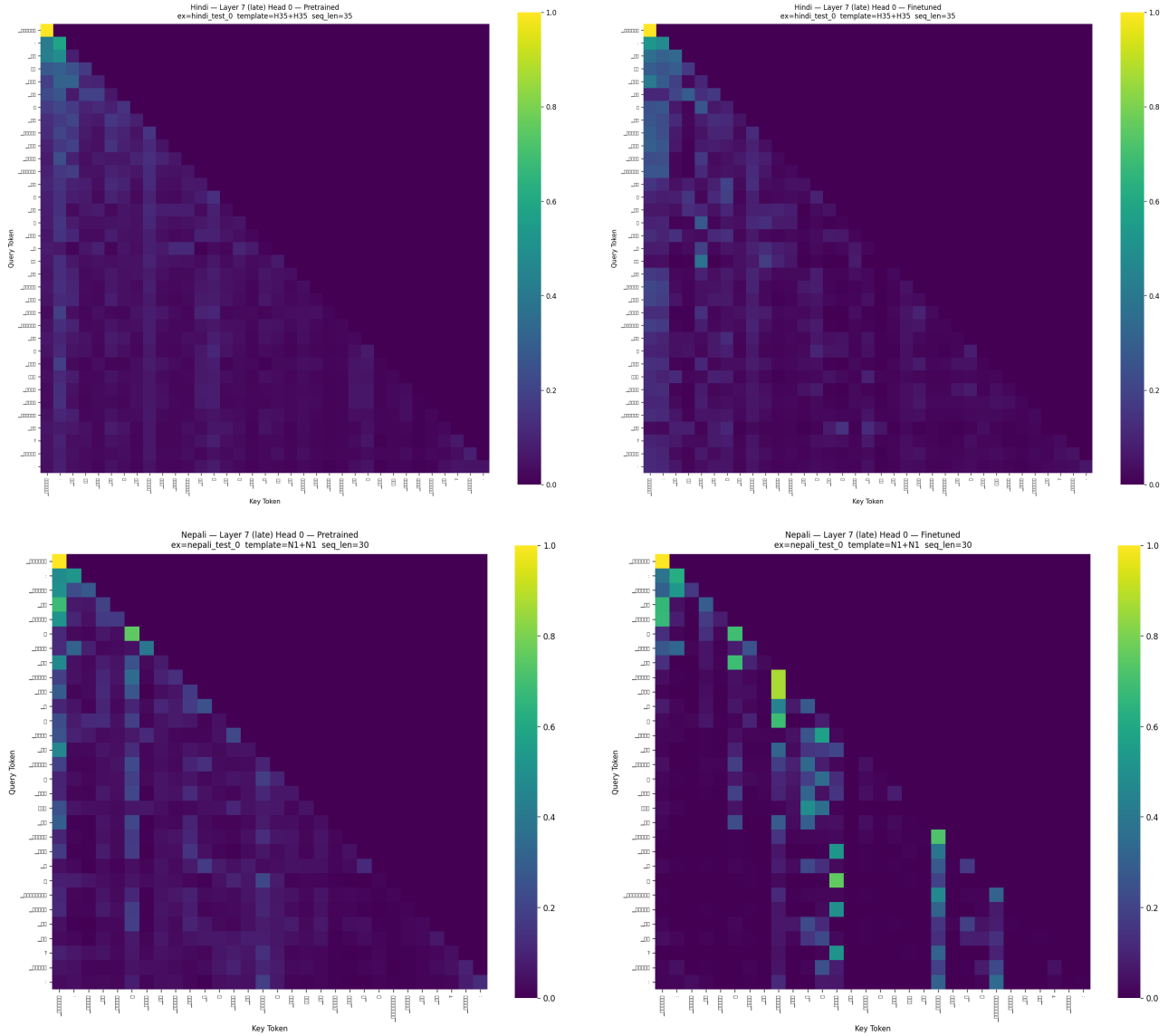


Figure 2: Phase 3 late-layer attention heatmaps comparing pretrained and fine-tuned models.

4.8 Model and Dataset Links

Fine-tuned models:

https://drive.google.com/drive/folders/1hGk6fKQAiFgree384PXe-Y-h3nwzsh1U?usp=sharing

Fine-tuning datasets:

https://drive.google.com/drive/folders/1qvZ75bkvMZW7_W4K7JvYFYLRQBqXbPzd?usp=sharing

5 Analysis and Discussion

5.1 1. Effect of model/data scale on reasoning performance

The model with around 25.8M parameters was able to learn the logic puzzles when they were given in familiar templates. This can be seen from the accuracy going up to around 75-76% for both languages after fine-tuning. However, when we tested the model on completely new template structures, the performance dropped a lot. This suggests that the model size and the amount of fine-tuning data used here were enough to learn the patterns present in the training data, but were not enough to achieve strong generalization to new reasoning structures.

5.2 2. Relationship between language/data quality and model performance

Even though Hindi and Nepali had different corpus sizes and different characteristics during pre-training, their familiar-template accuracies after fine-tuning were quite similar, with Hindi at 76.49% and Nepali at 75.29%. This shows that for this particular synthetic reasoning task, the structure of the fine-tuning dataset had a much larger effect on the final reasoning accuracy than the differences between the two pretrained language models.

5.3 3. Evidence for reasoning/generalization from familiar to held-out templates

The generalization to held-out templates was quite weak. While the models achieved around 75-76% accuracy on familiar templates, this dropped to 14.51% for Hindi and 7.32% for Nepali on held-out templates. This large difference suggests that the models were relying heavily on the patterns and structures they saw during fine-tuning, rather than learning reasoning rules that could be applied equally well to completely different template structures.

5.4 4. Influence of tokenizer/corpus characteristics

The results show that the tokenizers were able to represent the reasoning prompts well enough for the models to learn the task and produce the required answer tokens. Since the exact-match evaluation depends on correctly identifying the discrete answer, the SentencePiece tokenizers provided a suitable representation for these reasoning examples. However, the Phase 3 results alone are not enough to separate the exact effect of tokenizer characteristics from the effects of the pretrained models and the fine-tuning data.

6 Limitations

- **Training Duration:** The models were trained till 20,000 steps for the pretrained stage. The validation loss had already become fairly stable, but training for more steps could have potentially improved the language generation and other results.
- **Independent Models:** Both the Hindi and Nepali models were trained completely separately. We did not use any shared vocabulary or knowledge between them. A multilingual model could possibly have allowed some knowledge to transfer from the higher-resource Hindi language to the lower-resource Nepali language.
- **Evaluation Metrics:** One of the standard metrics we tried, ROUGE-L, did not work properly for our languages because the tokenizer used by the library was designed around Latin characters. It therefore removed the Devanagari characters and gave a score of 0 for all the generations. A language-appropriate tokenizer would be needed for this metric.
- **Synthetic Domain:** The reasoning evaluation was done completely on a synthetic dataset with a fixed number of templates. Therefore, the results should not be taken as a measure of how well the models can perform complex reasoning in real-world situations.
- **Dataset Size:** The fine-tuning dataset contained 9,600 training examples for each language. Although this was enough to get a large improvement on the familiar templates, it still gives the model limited exposure to different ways of expressing the same reasoning problem.

- **Generalization Gap:** There was a very large difference between the performance on familiar and held-out templates. This means that the high accuracy obtained on familiar examples does not necessarily mean that the model learned a general reasoning ability.
- **Metric Scope:** The exact-match metric only checks whether the required answer was correctly extracted from the model's output. It does not tell us whether the model generated a complete and logically coherent explanation or whether it would be able to perform unrestricted reasoning.
- **Attention Sample Size:** The attention analysis was performed on only 6 examples per language, consisting of 3 familiar and 3 held-out examples. Therefore, the attention results should be treated as a small analysis of how the models changed rather than as a general conclusion about the attention behaviour of the entire model.

7 Overall Conclusion

Overall, the three phases show how the models progressed from learning the basic language patterns from the collected corpora to being adapted for a specific reasoning task. Phase 1 showed the difference in the amount and composition of data available for Hindi and Nepali, while Phase 2 showed that both models were able to learn the language modelling task from their respective corpora. In Phase 3, fine-tuning produced a large improvement on the synthetic reasoning task for both languages, but the large drop on held-out templates showed that this improvement did not transfer equally well to new ways of expressing the same reasoning. The attention analysis also showed that the attention behaviour changed after fine-tuning, especially in the later layers. However, since the attention analysis was performed on a small number of examples, these changes should be treated as observations rather than proof of how the model performs reasoning.